

Eng : Youssef Ahmed

Section 3

Topics:

1. Analysis of single loop
 2. Analysis of nested loop (Multiple loops)
 3. Asymtjc notation
-

Analysis of single loop

Rules Of Analysis by using big O

$$T(n) = 5n^2 + n\log(n)$$

1. get the dominate term

$$T(n) = 5n^2$$

2. drop constant

$$T(n) = 5n^2 = O(n^2)$$

Case O(1)

```
for (int i = 0 ; i < c ; i= i+1)
{ // body start

    // some O(1) expressions
    //body end
}
```

here loop is running constant times and performing $O(1)$ at each operation as shown in body of for loop

$$O(loop) = c * O(1) = O(1) * O(1) = O(1)$$

Case O(n)

- Example 1

```
for (int i = 0 ; i < n ; i = i + 2)
{
    // some O(1) expressions
}
```

- assume n = 10

step	i
initial value	0
1	(0+2) = 2
2	(2+2) = 4
3	(4+2) = 6
4	(6+2) = 8
5	(8+2) = 10 stop !(i < n)

here the if user but n then loop will work 5 times that means :

$$n/c = n/2 = 10/2 = 5$$

- **Big O :**

$$T(n) = n/2 * O(1)$$

$$O(loop) = O(n)$$

increment by constant (C)

```
for (int i = 0 ; i < n ; i = i + c)
{
    // some O(1) expressions
}
```

here loop is running n / c times and performing $O(1)$ at each operation as shown in body of for loop

- **Big O :**

$$T(n) = n/c * O(1)$$

$$O(loop) = O(n)$$

Decrement by constant (C)

```
for (int i = n ; i > 0; i = i - c)
{
    // some O(1) expressions
}
```

here loop is running n / c times and performing $O(1)$ at each operation as shown in body of for loop

- **Big O :**

$$T(n) = n/c * O(1)$$

$$O(loop) = O(n)$$

Case Log (n)

- Example 1

```
for (int i = 1 ; i < n ; i = i * 2)
{
    // some O(1) expressions
}
```

- assume $n = 10$

i	iterations in body
$1 = 2^0$	1
$2 = 2^1$	1
$4 = 2^2$	1

i	iterations in body
$8 = 2^3$	1
$16 = 2^4$ stop ($i < n$) false	1

$2^0, 2^1, 2^2, \dots, 2^k$

$$n/2^k = 1$$

$$n = 2^k$$

$$\log(n) = k$$

- **Big O**

$$T(n) = \log_2(n) + 1$$

$$O(\text{loop}) = \log(n)$$

Multiple by Factor (C)

```
for (int i= 1 ; i< n; i = i * c)
{
    // some O(1) expressions
}
```

here loop is running n / c times and performing $O(1)$ at each operation as shown in body of for loop

- **Big O :**

$$T(n) = \log_c(n) * O(1)$$

$$O(\text{loop}) = O(\log(n))$$

Divide by constant (C)

```
for (int i= n ; i > 0; i = i / c)
{
    // some O(1) expressions
}
```

here loop is running n / c times and performing $O(1)$ at each operation as shown in body of for loop

- **Big O :**

$$T(n) = \log_c(n) * O(1)$$

$$O(\text{loop}) = O(\log(n))$$

Case $O(m + n)$

```
for (int i= 0 ; i < n; i = i + 1)
{
    // some O(1) expressions
}

for (int j= 0 ; j < n; i = j + 1)
{
    // some O(1) expressions
}
```

- **loop1**

$$O(\text{loop}) = O(n)$$

- **loop2**

$$O(\text{loop}) = O(m)$$

Total Big O

$$O(\text{Total}) = O(n) + O(m) = O(n + m)$$

Analysis of nested loop (Multiple loops)

Case (n²)

- Example

```
for(int i=0 ;i<n;i++)
{
    for(int j = 0; j<n ; j++)
    {
        // o(1)
    }
}
```

- First loop

$$O(\text{loop1}) = O(n)$$

- Second loop

$$O(\text{loop2}) = O(n)$$

- Total loop

$$O(\text{Total}) = O(n) * O(n) = O(n^2)$$

```
for(int i=0 ;i<n;i++)
{
    for(int j = 0; j<i ; j++)
    {
        // o(1)
    }
}
```

i	j	Hint
0 (initial)	0	-
1	0 , 1	-
2	0,1,2	-
3	0,1,2,3	-
-	-	-
n	0,1,2,3,4,.....,n	Arithmetic Sequence

$$\sum_{i=1}^n i = n(n+1)/2 = (n^2 + n)/2$$

$$O(Total) = (n^2 + n)/2 = O(n^2)$$

Type Of Classes

Class	Big O	Algorithms
Constant	1	math
logarithmic	$\log(n)$	Binary Search
linear	n	linear Search
Quadric	n^2	Selection Sort
Cubic	n^3	Sum of Tuples
Exponential	2^N	Fibonacci sequence in recursion

